

Практическая работа №6

Работа с файлами и директориями

Цель работы: получить практические навыки работы с файлами и директориями с помощью РНР.

Лабораторное оборудование: персональный компьютер с доступом в Интернет.

Время: 2 часа.

Краткие теоретические сведения

Файл – это именованная область данных на носителе информации, используемая как базовый объект взаимодействия с данными в операционных системах. Различают программные (исполняемые) файлы, управляющие работой компьютера, и файлы, содержащие прочие данные, например, текст, изображения и другое.

Последовательность работы с файлами в РНР:

- Открыть файл.
- Проверить на удачное (или неудачное) открытие файла.
- Обработать файл (чтение/запись).
- Закрыть файл.

Функция открытия файла

fopen – это функция, предназначенная для открытия файла. Она возвращает дескриптор файла. Определение функции: ***fopen(string \$filename, string \$mode)***. Первый параметр ***\$filename*** представляет путь к файлу, а второй – режим открытия.

Дескриптор файла – представляет собой указатель на открытый файл. Сам по себе дескриптор – это целое число, которое ОС использует для представления открытых файлов или другие объектов ввода/вывода.

Методы открытия файла

- «***r***» - открыть файл только для чтения; после открытия указатель файла устанавливается в начало файла;
- «***r+***» - открыть файл для чтения и записи; после открытия указатель файла устанавливается в начало файла;
- «***w***» - создать новый пустой файл только для записи; если файл с таким именем уже есть вся информация в нем уничтожается;
- «***w+***» - создать новый пустой файл для чтения записи; если файл с таким именем

уже есть вся информация в нем уничтожается;

- «a» - открыть файл для дозаписи; данные будут записываться в конец файла;
- «a+» - открыть файл для дозаписи и чтения данных; данные будут записываться в конец файла;
- «b» - флаг, указывающий на работу (чтение и запись) с двоичным файлом; указывается только в Windows.

Пример открытия файла приведен на рисунке 6.1.

```
$f1 = fopen("c:/www/html/file.txt", "r");  
if (!$f1)  
{ echo "Ошибка открытия файла"; }  
else { операции с файлом... }  
fclose($f1);
```

PHP

Рисунок 6.1 – Пример открытия файла в PHP

Обработка файла – это набор инструментов, которые позволяют выполнять операции с файлами, такие как создание новых, чтение существующих, обновление их содержимого и удаление. Обработка файлов является важной частью любого веб-приложения, так как часто нужно открывать и обрабатывать файл для разных задач.

Функции чтения данных из файла

- 1) Чтение строки из файла. Структура функции: *string fread(int file, int length)*

```
$fValue = fread($f1, filesize($f1));
```

PHP

Рисунок 6.2 – Пример использования функции чтения строки из файла

- 2) Чтение строки из уже открытого файла или до length байт из дескриптора файла. Структура функции: *string fgets(int file, [int length])*

```
$fValue = fgets($f1, 20);
```

PHP

Рисунок 6.3 – Пример использования функции побайтового чтения из файла

- 3) Обработка данных из файла в соответствии с форматом. Структура функции:

fscan(int file, string format)

```
$fValue = fscanf($handle, "%s\t%s\t%s\n");
```

 PHP

Рисунок 6.4 – Пример использования функции чтения данных из файла в соответствии с заданным форматом строки

Функции записи данных в файл

1) Запись строки в файл. Структура функции: *fputs(int file, int length)*

```
fputs($f1, "Hello, world");
```

 PHP

Рисунок 6.5 – Пример использования функции чтения строки из файла

2) Запись данных в открытый файл. Структура функции: *fwrite(int file, string str, int length)*

```
fwrite($f1, "This is some sample data");
```

 PHP

Рисунок 6.6 – Пример использования функции побайтового чтения из файла

3) Записывает отформатированную строку в файл. Структура функции: *fprintf(int file, string format)*

```
fprintf($f1, "There are $number million  
bicycles in $str");
```

 PHP

Рисунок 6.7 – Пример использования функции чтения данных из файла в соответствии с заданным форматом строки

Также существуют полезные функции, которые вам помогут при работе с файлами:

- Проверяет, достигнут ли конец файла. Функция: *feof(int file)*;
- Возвращает размер указанного файла (байт). Функция: *filesize(int file)*;
- Проверяет наличие указанного файла или каталога. Функция: *file_exists(int file)*;
- Проверяет время последнего доступа к файлу. Функция: *fileatime(int file)*.

Базовые примеры работа с чтением и записью данных в файл в PHP приведены на рисунках 6.8 и 6.9.

```
$filename = 'example.txt';
$file = fopen($filename, 'r');
if ($file) {
    $content = fread($file,
filesize($filename));
    fclose($file);
    echo $content;
} else {
    echo "Не удалось открыть
файл.";
}
```

Рисунок 6.8 – Пример работы с чтением файла в PHP

```
$filename = 'example.txt';
$file = fopen($filename, 'w');
if ($file) {
    $content = "Пример текста для
записи в файл.";
    fwrite($file, $content);
    fclose($file);
    echo "Данные успешно
записаны.";
} else {
    echo "Не удалось открыть
файл.";
}
```

Рисунок 6.9 – Пример работы с записью данных в файл в PHP

Для тех, кто молодец и читает методичку перед работой! Вот вам божеский подгон со скриптом, который позволит вам удалить все файлы из директории, а затем удалит и саму директорию:

```
<?php
// Путь к директории
$directory = "my_directory";
```

```
// Проверяем, существует ли директория
if (is_dir($directory)) {
    // Открываем директорию
    if ($handle = opendir($directory)) {
        // Читаем содержимое директории
        while (($file = readdir($handle)) !== false) {
            // Пропускаем специальные элементы "." и ".."
            if ($file != "." && $file != "..") {
                // Полный путь к файлу
                $filePath = $directory . "/" . $file;
                // Удаляем файл
                unlink($filePath);
                echo "Удалён файл: " . $filePath . "<br>";
            }
        }
        // Закрываем директорию
        closedir($handle);

        // Удаляем саму директорию
        if (rmdir($directory)) {
            echo "Директория " . $directory . " успешно удалена.";
        } else {
            echo "Не удалось удалить директорию " . $directory . ".";
        }
    } else {
        echo "Не удалось открыть директорию " . $directory . ".";
    }
} else {
    echo "Директория " . $directory . " не существует.";
}
?>
```

Работа с директориями

Директория – это структура файловой системы, которая организует и хранит файлы и другие директории. Также известная под привычным названием папка. PHP предоставляет функции для работы с директориями, такие как: открытие, чтение и закрытие. Вот самые основные функции для работы с директориями:

1) ***opendir()*** – это функция, которая открывает директорию по определённому пути. С её помощью можно просматривать файлы и каталоги внутри директории и выполнять различные операции с ними (рисунок 6.10).

```

$dir = 'path/to/directory';
$handle = opendir($dir);
if ($handle) {
    // Работаем с открытой директорией
    closedir($handle); // Закрываем директорию
} else {
    echo "Не удалось открыть директорию";
}

```

PHP

Рисунок 6.10 – Пример использования функции opendir()

2) **readdir()** – это функция, которая возвращает имя следующей по порядку элемента каталога. Элементы возвращаются в том порядке, в котором они хранятся в файловой системе (рисунок 6.11).

```

$dir = "/images/";
// Открываем директорию и читаем её содержимое
if (is_dir($dir)) {
    if ($dh = opendir($dir)) {
        while (($file = readdir($dh)) !== false) {
            echo "filename:" . $file . "<br>";
        }
    }
}

```

PHP

Рисунок 6.11 – Пример использования функции readdir()

3) **rmdir()** – это функция, которая удаляет директорию по указанному пути. При этом директория должна быть пустой и должны иметься необходимые для этого права (рисунок 6.12).

```

$dir = 'path/to/directory';
if (rmdir($dir)) {
    // Директория успешно удалена
} else {
    echo "Не удалось удалить директорию";
}

```

PHP

Рисунок 6.12 – Пример использования функции rmdir()

Задания на выполнение практической работы

Примечание. Для выполнения поставленной задачи рекомендуется использовать специальное программное обеспечение **Visual Studio Code (VS Code)** или аналог в виде

Sublime Text (ST). Для удобства разработки рекомендуется установить расширение **Live Server** by Ritwick Dey (идентификатор расширения: ritwickdey.LiveServer).

Также для выполнения поставленной задачи вам понадобится Open Server, он уже установлен на рабочих компьютерах, поэтому никаких действий по установке не требуется.

Задание 1. Найдите в поиске приложение «Open Server Panel» и запустите его (рисунок 6.13).

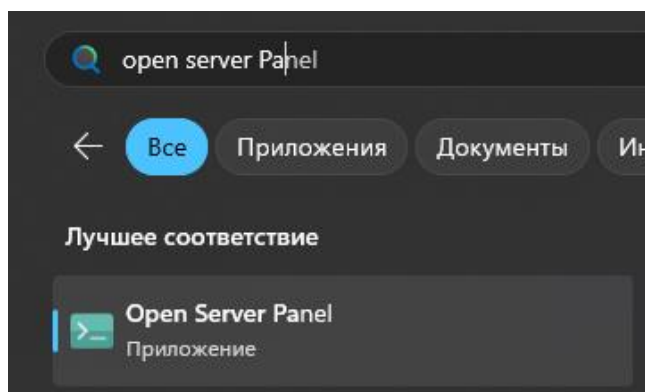


Рисунок 6.13 – Поиск и запуск приложения «Open Server Panel»

Задание 2. Если всё выполнено корректно, в трее вы увидите зеленый флажок – это и есть наш локальный сервер (Open Server Panel). Нажмите на него правой кнопкой мыши (ПКМ) и откройте папку с проектами (рисунок 6.14).

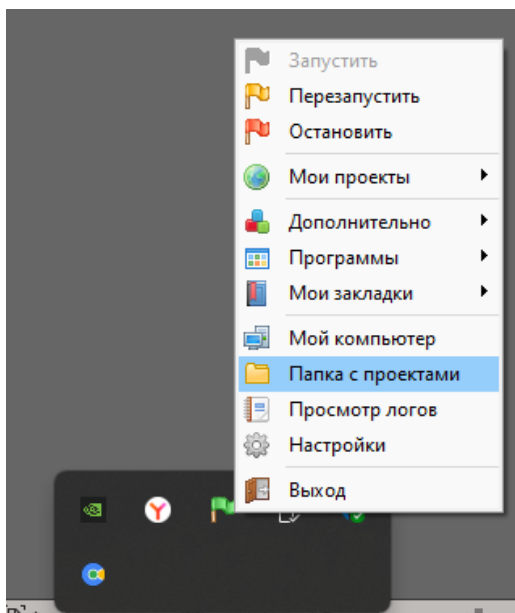


Рисунок 6.14 – Запущенный локальный сервер «Open Server Panel»

Задание 3. После открытия папки с проектами в проводнике Windows, создайте там папку, назовите её как-нибудь на английской раскладке, без пробелом. Например: *MySite*.

Задание 4. Скопируйте путь к данной папке в адресной строке проводника windows

как показано на рисунке 6.15.

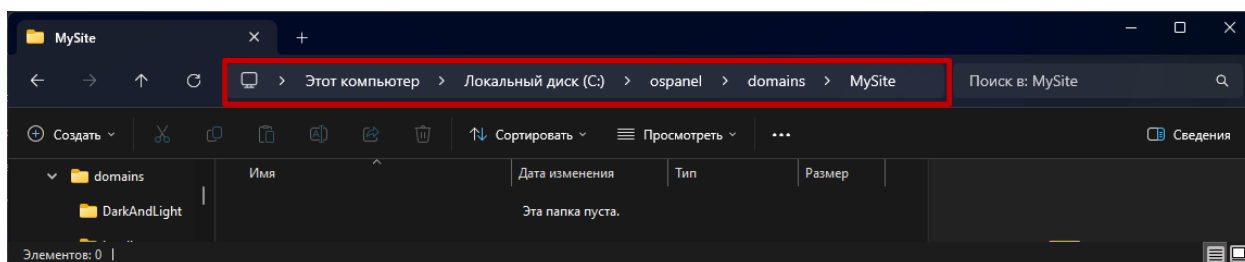


Рисунок 6.15 – Скопируйте путь к папке в проводнике Windows

Задание 5. Запустите программное обеспечение Visual Studio Code и выполните следующую команду: *файл → открыть папку (Ctrl + K + O)*. В открывшемся окне проводника Windows, вставьте в адресную строку путь, который скопировали в задании 4.

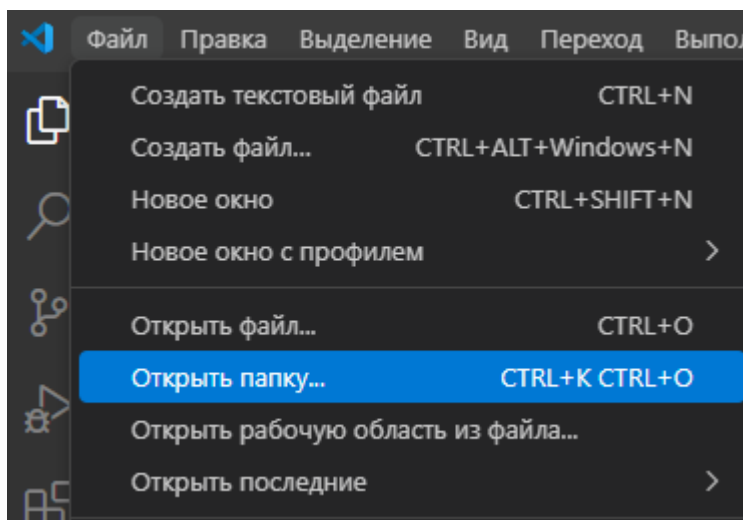


Рисунок 6.16 – Выполнение команды файл → открыть папку

*Отлично, у вас всё получилось! Теперь эта папка – ваш маленький проект, который будет работать на локальном сервере, теперь мы можем работать с файлами расширения *.php и открывать их в браузере.*

Задание 6. Согласно тематики вашего сайта, необходимо написать PHP-скрипт, который спрашивает у пользователя его имя и фамилию, а также пользователь выбирает из списка товар/услуг (по крайней мере 3-5 товаров/услуг). После успешного заполнения всех данных, занесите их в файл *order.txt* (файл должен создаваться автоматически). После корректно записи данных в файл, прочитайте его и выведите его содержимое на странице.

Задание 7. Доработайте предыдущее задание: теперь для каждого «заказа

товара/услуги», должен создаваться отдельный файл. Для этого воспользуйтесь фамилией пользователя в качестве уникального имени файла. Продемонстрируйте динамическое создание файлов по крайней мере для 5 «заказов товара/услуги».

Задание 8. Отлично, теперь вы можете генерировать отдельный файл для каждого заказа! Но было бы здорово, если бы вы динамически создали директорию для этих файлов, назвав её, например, *orders*. В данной директории, также динамически (с помощью PHP-скрипта), создайте столько папок, сколько у вас написано товаров/услуг на сайте. В качестве имени папки дайте имя самого товара/услуги.

Примечание: предусмотрите то, что пользователь глупый и нечаянно зашел, и удалил все ваши папки. Отсюда следует следующее: проверяйте наличие папок в файловой системе, если папку удалили, и она не найдена – создайте новую, **и так со всеми папками**.

Задание 9. В нашей файловой системе нужно сохранять порядок. Поэтому доработайте ваш код по следующему описанию: теперь, создаваемый файл по фамилии пользователя должен попадать именно в ту папку в файловой системе, которая соответствует названию выбранного товара/услуги при заполнении данных на сайте.

Задание 10. Пора тотально всё удалить! Поэтому, создайте PHP скрипт, который удалит все вложенные файлы и папки до единой! Для этого, воспользуйтесь соответствующими функциями для работы с директориями и файлами и циклами в PHP!

Подсказка: для удаления файлов воспользуйтесь функцией *unlink()*, а для удаления директории функцией *rmdir()*. Используйте циклы *foreach* и *while* для упрощения работы с удалением файлов и директорий.

Контрольные вопросы

1. Какая функция используется для открытия файла в PHP? Какие режимы открытия файла вы знаете?
2. Как прочитать содержимое файла в PHP? Приведите пример использования функции *fread()*.
3. Как создать директорию в PHP? Какая функция используется для этого?
4. Как удалить файл или директорию в PHP? В чём разница между *unlink()* и *rmdir()*?
5. Как прочитать содержимое директории в PHP? Приведите пример использования функции *readdir()*.

Содержание отчёта

Отчет выполняется единым, в текстовом (электронном) виде, который содержит номер лабораторной/практической работы, тему и цель работы, а также краткое описание

выполняемых операций и скриншоты результатов выполнения, ответы на контрольные вопросы, выводы. Отчёт оформляется согласно ГОСТ 7.32-2017.

Структура отчёта должна содержать следующее:

- наименование и номер работы;
- тему и цель работы;
- раздел «Постановка задачи», где описываются задачи, которые необходимо решить в ходе выполнения практической работы;
- раздел «Ход работы», где описывается ход выполнения работы с кратким описанием выполняемых операций и приведенных результатов выполнения в виде скриншотов;
- листинг программы (при наличии, в зависимости от поставленной задачи);
- ответы на контрольные вопросы;
- выводы (приводятся исходя из полученных знаний и выполненных работ, **указание выводов основываясь только из поставленной цели работы не допускается!**).